Integer Data Representation

Integers

- Integers in computer systems represent discrete whole numbers using binary encoding schemes.
- The representation method determines both the range of values and the arithmetic operations possible within a fixed bit width.

Unsigned Integers

Unsigned integers **dedicate all available bits to magnitude representation** using standard binary positional notation. This maximizes the unsigned nonnegative range but by proxy, does not allow negative number representation.

> Example 1: $+42_{10}$ into 8-bit Unsigned Integer
>
> 42 = 0010 1010
>
> result = 0010 1010 / 0x2A

> Example 2: -82_{10} into 8-bit Unsigned Integer
>
> negative number as unsigned int -> invalid
>
> result = N/A

Zero-Extension Process

When converting from smaller to larger representations, the original value is preserved by **padding the most significant positions with zeros**.

> Example 1: Zero extend 0xF7 into a 16-bit Unsigned Integer
>
> 0xF7 = 1111 0111
>
> 0000 0000 1111 0111 (pad zeroes until you reach desired width)
>
> result = 0000 0000 1111 0111 / 0x00F7

Signed Integers

Signed integer representation has evolved through several encoding schemes, each addressing specific limitations of predecessors while introducing new trade-offs.

Sign-and-Magnitude System

The sign-and-magnitude system attempts to represent numbers in an **intuitive and human-readable format**. It has a symmetric positive/negative range and has dual representations for zero (positive and negative 0).

Basic Steps:

1. **Convert**: Get magnitude and store it as an unsigned integer with n - 1 bits (n being the total number of bits).
    - If it cannot be stored in n - 1 bits, automatically out of bounds (since MSbit is sign bit).
2. **Sign**: Set MSbit to the sign of the number (positive = 0, negative = 1).

> Example 1: $+42_{10}$ into 8-bit Signed Integer (S&M)
>
> 42 = 010 1010
> sign bit = 0
>
> result = 0010 1010 / 0x2A

> Example 2: -82_{10} into 8-bit Signed Integer (S&M)
>
> 82 = 101 0010
> sign bit = 1
>
> result = 1101 0010 / 0xD2

> Example 3: $+128_{10}$ into 8-bit Signed Integer (S&M)
>
> 128 = 1000 0000 -> out of bounds (not in range)
>
> result = N/A

One's Complement System

Implementing 1's Complement **only requires a simple negation operation**, meaning that it maintains a bit pattern that is easily manipulated. However, it still has the problem with having dual representations for zero, an end-around carry required for arithmetic, and a non-contiguous value representation.

Basic Steps:

1. **Convert**:
    - If positive, use the same method as unsigned integers.
    - If negative, perform a bitwise NOT on the unsigned integer representation.
    - If it cannot be stored in n - 1 bits, automatically out of bounds (since MSbit is sign bit).

> Example 1: $+42_{10}$ into 8-bit Signed Integer (1's C)
>
> 42 = 0010 1010
>
> result = 0010 1010 / 0x2A

> Example 2: -82_{10} into 8-bit Signed Integer (1's C)
>
> 82 = 0101 0010
> bitwise NOT = 1010 1101 (since negative)
>
> result = 1010 1101 / 0xAD

> Example 3: $+128_{10}$ into 8-bit Signed Integer (1's C)
>
> 128 = 1000 0000 -> out of bounds (not in range)
>
> result = N/A

Two's Complement System

2's solves the problems of 1's Complement by being a format that fulfills the following: (1) a single zero representation, (2) a contiguous value space, (3) working directly with standard binary arithmetic, and (4) An asymmetric range that provides an additional negative value.

Basic Steps:

1. **Convert**: Use the same method as 1's Complement. The only exception is that 2's C has a representation for -128.
2. **Add**: If negative, increment the result to get its 2's Complement. Overflow is ignored.

> Example 1: $+42_{10}$ into 8-bit Signed Integer (2's C)
>
> 42 = 0010 1010
>
> result = 0010 1010 / 0x2A

> Example 2: -128_{10} into 8-bit Signed Integer (2's C)
>
> 128 = 1000 0000
> bitwise NOT = 0111 1111 (since negative)
> increment = 1000 0000 (since negative)
>
> result = 1000 0000 / 0x80

> Example 3: $+128_{10}$ into 8-bit Signed Integer (2's C)
>
> 128 = 1000 0000 -> out of bounds (not in range)
>
> result = N/A

Sign Extension in Two's Complement

Used to maintain the two's complement value by **preserving the weighted contribution of the sign bit** across different widths.

Basic Steps:

1. **Extend**:
   - Positive numbers: MSb = 0, extend with 0s.
   - Negative numbers: MSb = 1, extend with 1s.
   - **Tip**: Notice that you simply have to pad it using the MSb.

CSARCH2: Computer Organization & Architecture 2

Number Systems and Data Representation

Integer Data Cheatsheet

Bits	Format	Lower Bound	Upper Bound
8	Unsigned	0	255
	S&M	-127	+127
	1's C	-127	+127
	2's C	-128	+127

Bits	Format	Lower Bound	Upper Bound
16	Unsigned	0	+65 535
	S&M	-32 767	+32 767
	1's C	-32 767	+32 767
	2's C	-32 768	+32 767

Bits	Format	Lower Bound	Upper Bound
32	Unsigned	0	$2^{32} - 1$
	S&M	$-2^{32-1} + 1$	$2^{32-1} - 1$
	1's C	$-2^{32-1} + 1$	$2^{32-1} - 1$
	2's C	-2^{32-1}	$2^{32-1} - 1$

Bits	Format	Lower Bound	Upper Bound
64	Unsigned	0	$2^{64} - 1$
	S&M	$-2^{64-1} + 1$	$2^{64-1} - 1$
	1's C	$-2^{64-1} + 1$	$2^{64-1} - 1$
	2's C	-2^{64-1}	$2^{64-1} - 1$

Data Type	Bits	Format Specifier	
		Unsigned	Signed
char	8	`%hhu`	`%hhd`
short	16	`%hu`	`%hd`
int	32	`%u`	`%d`
single-precision float	32	-	`%f`
long long int	64	`%llu`	`%lld`
double-precision float	64	-	`%lf`